

Batch Methods

Master Degree in Control Systems Engineering 2024-2025
University of Padova

Teachers: Prof. Ruggero Carli & Prof. Gian Antonio Susto

Second part of Today's lecture

- Batch methods
- Linear Least-squares Predictions

Batch methods in Reinforcement Learning

- Gradient descent is simple and appealing
- But it is **not** sample efficient
- Batch methods seek to find the best fitting value function
- Given the agent's experience (**training data**)

Intuition behind batch methods: *in general one of the requirements for SGD optimization is that the training data is **independent and identically distributed** and when an Agent interacts with the environment, the sequence of experience tuples can be highly correlated.*



*Learning algorithms that learn from each of these experiences tuples in **sequential order** runs the risk of getting swayed by the effects of this correlation*

Experience Replay

We can improve this situation *using a large buffer of past experience and sample training data from it (a random subset)*, instead of using latest experience

This technique is called *replay buffer* or *experience buffer*. The replay buffer contains a collection of tuples (S, A, R, S') . The tuples are gradually added to the buffer as we are interacting with the environment

Experience Replay

We can improve this situation *using a large buffer of past experience and sample training data from it (a random subset)*, instead of using latest experience

This technique is called *replay buffer* or *experience buffer*. The replay buffer contains a collection of tuples (S, A, R, S') . The tuples are gradually added to the buffer as we are interacting with the environment

The simplest implementation is a buffer of fixed size, with new data added to the end of the buffer so that it pushes the oldest experience out of it

The act of sampling a small batch of tuples from the replay buffer in order to learn is known as *experience replay*

Experience Replay

We can improve this situation *using a large buffer of past experience and sample training data from it (a random subset)*, instead of using latest experience

This technique is called *replay buffer* or *experience buffer*. The replay buffer contains a collection of tuples (S, A, R, S') . The tuples are gradually added to the buffer as we are interacting with the environment

The simplest implementation is a buffer of fixed size, with new data added to the end of the buffer so that it pushes the oldest experience out of it

The act of sampling a small batch of tuples from the replay buffer in order to learn is known as *experience replay*

In addition to breaking harmful correlations, experience replay allows us to learn more from individual tuples multiple times, recall rare occurrences, and in general make better use of our experience

Least squares Prediction

- Given value function approximation $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$
- and experience $\mathcal{D}$ consisting of $\langle state, value \rangle$ pairs

$$\mathcal{D} = \{ \langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle \}$$

- which parameters $\mathbf{w}$ give the *best fitting* value function $\hat{v}(s, \mathbf{w})$
- **Least squares** algorithms find parameter vector $\mathbf{w}$ minimising sum-squared error between $\hat{v}(s_t, \mathbf{w})$ and v_t^π

$$\begin{aligned} LS(\mathbf{w}) &= \sum_{t=1}^T (v_t^\pi - \hat{v}(s_t, \mathbf{w}))^2 \\ &= \mathbb{E}_{\mathcal{D}} \left[(v^\pi - \hat{v}(s, \mathbf{w}))^2 \right] \end{aligned}$$

Stochastic Gradient Descent with Experience Replay

Given experience $\mathcal{D}$ consisting of $\langle state, value \rangle$ pairs

$$\mathcal{D} = \{ \langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle \}$$

Repeat :

1. Sample state, value from experience

$$\langle s, v^\pi \rangle \sim \mathcal{D}$$

2. Apply stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha (v^\pi - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$$

Linear Least Squares Prediction Algorithms

- We do not know true values v_t^π
- In practice, our *training data* must use noisy or biased samples of v_t^π

LSMC Least Squares Monte-Carlo uses return

$$v_t^\pi \approx G_t$$

LSTD Least Squares Temporal-Difference uses TD target

$$v_t^\pi \approx R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$$

LSTD(λ) Least Squares TD(λ) uses λ -return

$$v_t^\pi \approx G_t^\lambda$$

Linear Least Squares Prediction

- Experience replay finds least squares solution
- But it may take many iterations
- Using **linear** value function approximation $\hat{v}(s, \mathbf{w}) = \mathbf{x}(s)^T \mathbf{w}$
- We can solve the least squares solution directly

Linear Least Squares Prediction

- At minimum of $LS(\mathbf{w})$, the expected update must be zero

$$\mathbb{E}_{\mathcal{D}} [\Delta \mathbf{w}] = 0$$

$$\Delta \mathbf{w} = \alpha (v^{\pi} - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

$$\alpha \sum_{t=1}^T \mathbf{x}(s_t) (v_t^{\pi} - \mathbf{x}(s_t)^T \mathbf{w}) = 0$$

Linear Least Squares Prediction

- At minimum of $LS(\mathbf{w})$, the expected update must be zero

$$\mathbb{E}_{\mathcal{D}} [\Delta \mathbf{w}] = 0$$

$$\Delta \mathbf{w} = \alpha (v^{\pi} - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

$$\alpha \sum_{t=1}^T \mathbf{x}(s_t) (v_t^{\pi} - \mathbf{x}(s_t)^T \mathbf{w}) = 0$$

$$\sum_{t=1}^T \mathbf{x}(s_t) v_t^{\pi} = \sum_{t=1}^T \mathbf{x}(s_t) \mathbf{x}(s_t)^T \mathbf{w}$$

Linear Least Squares Prediction

- At minimum of $LS(\mathbf{w})$, the expected update must be zero

$$\mathbb{E}_{\mathcal{D}} [\Delta \mathbf{w}] = 0$$

$$\Delta \mathbf{w} = \alpha(v^\pi - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

$$\alpha \sum_{t=1}^T \mathbf{x}(s_t) (v_t^\pi - \mathbf{x}(s_t)^T \mathbf{w}) = 0$$

$$\sum_{t=1}^T \mathbf{x}(s_t) v_t^\pi = \sum_{t=1}^T \mathbf{x}(s_t) \mathbf{x}(s_t)^T \mathbf{w}$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(s_t) \mathbf{x}(s_t)^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(s_t) v_t^\pi$$

Linear Least Squares Prediction

- At minimum of $LS(\mathbf{w})$, the expected update must be zero

$$\mathbb{E}_{\mathcal{D}} [\Delta \mathbf{w}] = 0$$

$$\Delta \mathbf{w} = \alpha(v^{\pi} - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

$$\alpha \sum_{t=1}^T \mathbf{x}(s_t) (v_t^{\pi} - \mathbf{x}(s_t)^T \mathbf{w}) = 0$$

$$\sum_{t=1}^T \mathbf{x}(s_t) v_t^{\pi} = \sum_{t=1}^T \mathbf{x}(s_t) \mathbf{x}(s_t)^T \mathbf{w}$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(s_t) \mathbf{x}(s_t)^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(s_t) v_t^{\pi}$$

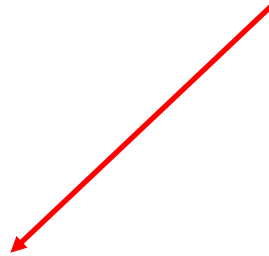
- For N features, direct solution time is $O(N^3)$
- Incremental solution time is $O(N^2)$ using Sherman-Morrison

Linear Least Squares Prediction

Shermann – Morrison Formula $(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1}u}$

Linear Least Squares Prediction

Shermann – Morrison Formula $(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1}u}$



$$\left(\sum_{t=1}^T \mathbf{x}(s_t) \mathbf{x}(s_t)^T \right)^{-1} = \left(\sum_{t=1}^{T-1} \mathbf{x}(s_t) \mathbf{x}(s_t)^T + \mathbf{x}(s_T) \mathbf{x}(s_T)^T \right)^{-1}$$

Linear Least Squares Prediction Algorithms

LSMC

$$0 = \sum_{t=1}^T \alpha (G_t - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t) \mathbf{x}(S_t)^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) G_t$$

LSTD

$$0 = \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t) (\mathbf{x}(S_t) - \gamma \mathbf{x}(S_{t+1}))^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) R_{t+1}$$

LSTD(λ)

$$0 = \sum_{t=1}^T \alpha \delta_t E_t$$

$$\mathbf{w} = \left(\sum_{t=1}^T E_t (\mathbf{x}(S_t) - \gamma \mathbf{x}(S_{t+1}))^T \right)^{-1} \sum_{t=1}^T E_t R_{t+1}$$

Convergence of Linear Least Squares Prediction Algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	LSMC	✓	✓	-
	TD	✓	✓	✗
	LSTD	✓	✓	-
Off-Policy	MC	✓	✓	✓
	LSMC	✓	✓	-
	TD	✓	✗	✗
	LSTD	✓	✓	-

Least Squares Action-Value Function Approximation

- Approximate action – value function $q_{\pi}(s, a)$
- using linear combination of features $\mathbf{x}(s, a)$

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^T \mathbf{w} \approx q_{\pi}(s, a)$$

Least Squares Action-Value Function Approximation

- Approximate action – value function $q_\pi(s, a)$
- using linear combination of features $\mathbf{x}(s, a)$

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^T \mathbf{w} \approx q_\pi(s, a)$$

- Minimise *least-squares error* between $\hat{q}(s, a, \mathbf{w})$ and $q_\pi(s, a)$
- from *experience generated using policy π*
- consisting of $\langle (state, action), value \rangle$ pairs

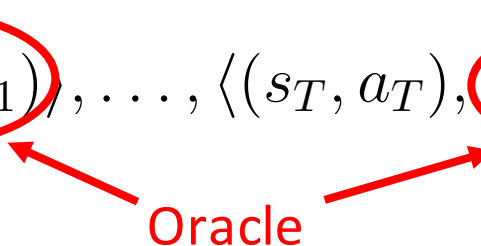
$$\mathcal{D} = \{ \langle (s_1, a_1), q_\pi(s_1, a_1) \rangle, \dots, \langle (s_T, a_T), q_\pi(s_T, a_T) \rangle \}$$

Least Squares Action-Value Function Approximation

- Approximate action – value function $q_\pi(s, a)$
- using linear combination of features $\mathbf{x}(s, a)$

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^T \mathbf{w} \approx q_\pi(s, a)$$

- Minimise *least-squares error* between $\hat{q}(s, a, \mathbf{w})$ and $q_\pi(s, a)$
- from *experience generated using policy π*
- consisting of $\langle (state, action), value \rangle$ pairs

$$\mathcal{D} = \{ \langle (s_1, a_1), q_\pi(s_1, a_1) \rangle, \dots, \langle (s_T, a_T), q_\pi(s_T, a_T) \rangle \}$$


The diagram shows the word "Oracle" in red text at the bottom. Two red arrows point from "Oracle" to the two circled terms in the set $\mathcal{D}$: $q_\pi(s_1, a_1)$ and $q_\pi(s_T, a_T)$. These terms are circled in red in the original image.

Least Squares Action-Value Function Approximation

- Approximate action – value function $q_\pi(s, a)$
- using linear combination of features $\mathbf{x}(s, a)$

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^T \mathbf{w} \approx q_\pi(s, a)$$

- Minimise *least-squares error* between $\hat{q}(s, a, \mathbf{w})$ and $q_\pi(s, a)$
- from *experience generated using policy π*
- consisting of $\langle (state, action), value \rangle$ pairs

$$\mathcal{D} = \{ \langle (s_1, a_1), q_\pi(s_1, a_1) \rangle, \dots, \langle (s_T, a_T), q_\pi(s_T, a_T) \rangle \}$$

The diagram shows the training data set $\mathcal{D}$ as a set of pairs $\langle (s, a), q_\pi(s, a) \rangle$. The values $q_\pi(s_1, a_1)$ and $q_\pi(s_T, a_T)$ are circled in red. Two red arrows point from the word "Oracle" below to these two circled values, indicating that the Oracle provides the target values for the training data.

Stochastic Gradient Descent

Least Squares Action-Value Function Approximation

Given the experience consisting of $\langle (state, action), value \rangle$ pairs

$$\mathcal{D} = \{ \langle (s_1, a_1), q_\pi(s_1, a_1) \rangle, \dots, \langle (s_T, a_T), q_\pi(s_T, a_T) \rangle \}$$

Repeat :

1. Sample state, value from experience

$$\langle (s, a), q_\pi(s, a) \rangle$$

2. Apply stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha (q_\pi(s, a) - \hat{q}(s, a, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s, a, \mathbf{w})$$

Least Squares Action-Value Function Approximation

Given the experience consisting of $\langle (state, action), value \rangle$ pairs

$$\mathcal{D} = \{ \langle (s_1, a_1), q_\pi(s_1, a_1) \rangle, \dots, \langle (s_T, a_T), q_\pi(s_T, a_T) \rangle \}$$

Repeat :

1. Sample state, value from experience

$$\langle (s, a), q_\pi(s, a) \rangle$$

2. Apply stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha (q_\pi(s, a) - \hat{q}(s, a, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s, a, \mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$$

Least Squares Action-Value Function Approximation

Stochastic gradient with TD target

$$\Delta \mathbf{w} = \alpha (\underline{q_\pi(s, a)} - \hat{q}(s, a, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s, a, \mathbf{w})$$



$$\Delta \mathbf{w} = \alpha (\underline{R_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w})} - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

Least Squares Action-Value Function Approximation

Stochastic gradient with TD target

$$\Delta \mathbf{w} = \alpha \left(\underline{q_\pi(s, a)} - \hat{q}(s, a, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{q}(s, a, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha \left(\underline{R_{t+1} + \gamma \hat{q}(s_{t+1}, \textcolor{red}{a}_{t+1}, \mathbf{w})} - \hat{q}(s_t, a_t, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

$$a_{t+1} \sim \pi(\cdot | S_{t+1})$$

Least Squares Action-Value Function Approximation

Stochastic gradient with TD target

$$\Delta \mathbf{w} = \alpha \left(\underline{q_\pi(s, a)} - \hat{q}(s, a, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{q}(s, a, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha \left(\underline{R_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w})} - \hat{q}(s_t, a_t, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

$$a_{t+1} \sim \pi(\cdot | S_{t+1})$$

Observation : $\mathcal{D}$ is a batch of tuples (S, R, A, S')

Least Squares Action-Value Function Approximation

Stochastic gradient with TD target

Given the experience consisting of visited tuples

$$\mathcal{D} = \{ \dots \langle (S, R, A, S') \rangle \dots \}$$

Repeat :

1. Sample tupla from experience

$$(S, R, A, S')$$

2. Apply stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha (R_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$$

Least Squares Q-Learning (LSTDQ) –Linear case

- Consider the following linear Q-learning update

$$\delta = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha \delta \mathbf{x}(S_t, A_t)$$


$$A_{t+1} \sim \pi(\cdot | S_{t+1})$$

Least Squares Q-Learning (LSTDQ) –Linear case

- Consider the following linear Q-learning update

$$\delta = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha \delta \mathbf{x}(S_t, A_t)$$

- LSTDQ algorithm : solve for total update = zero

$$0 = \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})) \mathbf{x}(S_t, A_t)$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t, A_t) (\mathbf{x}(S_t, A_t) - \gamma \mathbf{x}(S_{t+1}, A_{t+1}))^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t, A_t) R_{t+1}$$

Least Squares Q-Learning (LSTDQ) –Linear case

- Consider the following linear Q-learning update

$$\delta = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$

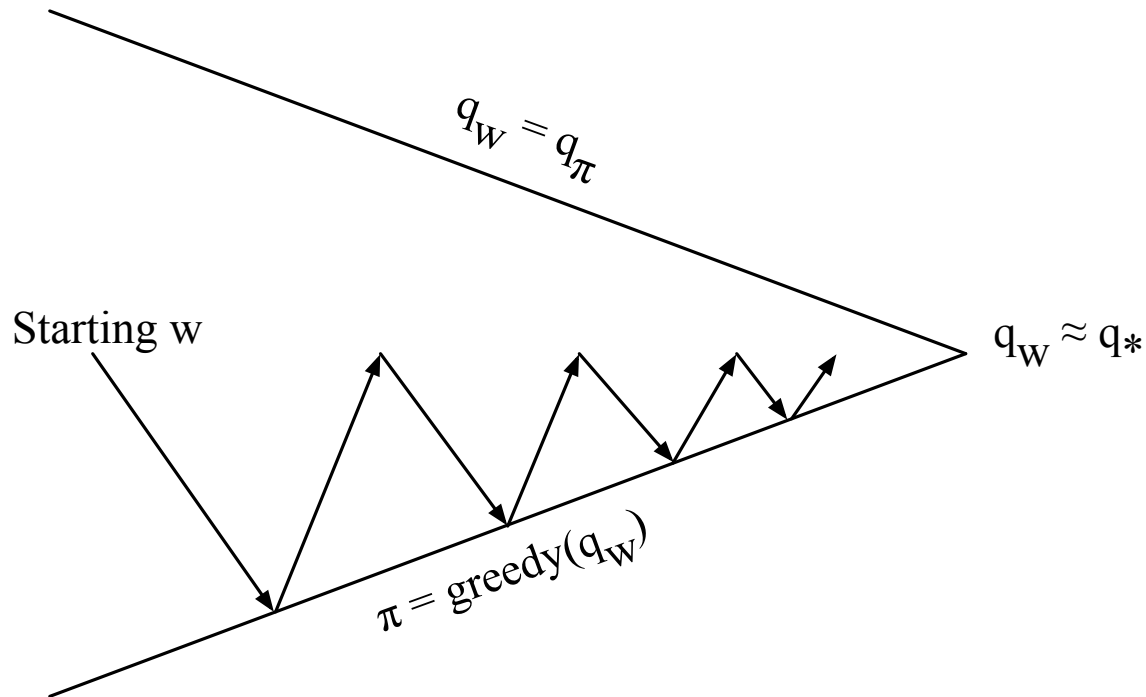
$$\Delta \mathbf{w} = \alpha \delta \mathbf{x}(S_t, A_t)$$

- LSTDQ algorithm : solve for total update = zero

$$0 = \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})) \mathbf{x}(S_t, A_t)$$
$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t, A_t) (\mathbf{x}(S_t, A_t) - \gamma \mathbf{x}(S_{t+1}, A_{t+1}))^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t, A_t) R_{t+1}$$

LSTDQ($\pi, \mathcal{D}$) provides estimate of action-value function
for a given policy π

Least Squares Control - Policy Iteration



Policy evaluation Policy evaluation by **least squares Q-learning**

Policy improvement Geedy policy improvement

Least Squares Control – Policy Iteration

...we use the same idea as Q-learning...

$$\delta = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$



$$A_{t+1} = \operatorname{argmax}_a \hat{q}(S_{t+1}, a, \mathbf{w})$$

Least Squares Control – Policy Iteration

...we use the same idea as Q-learning...

$$\delta = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$



$$A_{t+1} = \operatorname{argmax}_a \hat{q}(S_{t+1}, a, \mathbf{w})$$

- For policy evaluation, we want to efficiently use all experience
- For control, we also want to improve the policy
- We use the same idea as Q-learning
 - *Use experience generated by old policy*
 $S_t, A_t, R_{t+1}, S_{t+1} \sim \pi_{\text{old}}$
 - *Consider alternative successor action* $A' = \pi_{\text{new}}(S_{t+1})$
 - *Update* $\hat{q}(S_t, A_t, \mathbf{w})$ *towards value of alternative action*
 $R_{t+1} + \gamma \hat{q}(S_{t+1}, A', \mathbf{w})$

Least Squares Control – Policy Iteration

Stochastic gradient with TD target

Given the experience consisting of visited tuples

$$\mathcal{D} = \{ \dots \langle (S, R, A, S') \rangle \dots \}$$

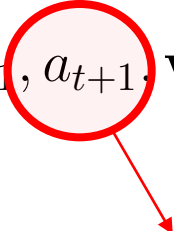
Repeat :

1. Sample tuple from experience

$$(S, R, A, S')$$

2. Apply stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha (R_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$


$$A_{t+1} = \operatorname{argmax}_a \hat{q}(S_{t+1}, a, \mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$$

Least Squares Q-Learning (LSTDQ) –Linear case

- Consider the following linear Q-learning update

$$\delta = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha \delta \mathbf{x}(S_t, A_t)$$


$$A_{t+1} = \operatorname{argmax}_a \hat{q}(S_{t+1}, a, \mathbf{w})$$

Least Squares Q-Learning (LSTDQ) –Linear case

- Consider the following linear Q-learning update

$$\delta = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha \delta \mathbf{x}(S_t, A_t)$$

- LSTDQ algorithm : solve for total update = zero

$$0 = \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})) \mathbf{x}(S_t, A_t)$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t, A_t) (\mathbf{x}(S_t, A_t) - \gamma \mathbf{x}(S_{t+1}, A_{t+1}))^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t, A_t) R_{t+1}$$

Least Squares Policy Iteration Algorithm

- The following pseudocode uses LSTDQ for policy evaluation
- It repeatedly re-evaluates experience $\mathcal{D}$ with different policies

ALGORITHM LSPI-TD($\mathcal{D}, \pi_0$)

$\pi' \leftarrow \pi_0$

repeat

$\pi \leftarrow \pi'$

$Q \leftarrow \text{LSTDQ}(\pi, \mathcal{D})$

for all $s \in \mathcal{S}$ **do**

$\pi'(s) \leftarrow \underset{a \in \mathcal{A}}{\operatorname{argmax}} Q(s, a)$

end for

until $(\pi \approx \pi')$

return π

Convergence of Control Algorithm

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	✓	(✓)	✗
Sarsa	✓	(✓)	✗
Q-learning	✓	✗	✗
LSPI	✓	(✓)	-

(✓) = chatters around near-optimal value function

Experience replay in Deep Q-networks (DQN)

Initialize network $Q(\theta)$

Initialize target network $\hat{Q}(\hat{\theta})$

Initialize experience replay memory D

Initialize the *Agent* to interact with the Environment

while *not converged* **do**

/ Sample phase*

$\epsilon \leftarrow$ setting new epsilon with ϵ -decay

 Choose an action a from state s using policy ϵ -greedy(Q) (based on $Q(s, a, \theta)$)

Agent takes action a , observe reward r , and next state s'

 Store transition $(s, a, r, s', done)$ in the experience replay memory D

if *enough experiences in D* **then**

/ Learn phase*

 Sample a random *minibatch* of N transitions from D

for *every transition* $(s_i, a_i, r_i, s'_i, done_i)$ *in minibatch* **do**

if $done_i$ **then**

$y_i = r_i$

else

$y_i = r_i + \gamma \max_{a'} \hat{Q}(s'_i, a', \hat{\theta})$

end

end

 Calculate the loss $\mathcal{L} = \mathbb{E}_D [y_j - Q(s_j, a_j, \theta)] = 1/N \sum_{i=1}^N (y_i - Q(s_i, a_i, \theta))^2$

 Update Q using the SGD algorithm by minimizing the loss $\mathcal{L}$ (update θ)

 Every C steps, copy weights from Q to $\hat{Q}$ ($\theta \rightarrow \hat{\theta}$)

end

end

Experience replay in Deep Q-networks (DQN)

Initialize network $Q(\theta)$

Initialize target network $\hat{Q}(\hat{\theta})$

Initialize experience replay memory D

Initialize the *Agent* to interact with the Environment

while *not converged* do

 /* Sample phase

$\epsilon \leftarrow$ setting new epsilon with ϵ -decay

 Choose an action a from state s using policy ϵ -greedy(Q) (based on $Q(s, a, \theta)$)

 Agent takes action a , observe reward r , and next state s'

 Store transition $(s, a, r, s', done)$ in the experience replay memory D

 if *enough experiences in D* then

 /* Learn phase

 Sample a random *minibatch* of N transitions from D

 for *every transition* $(s_i, a_i, r_i, s'_i, done_i)$ in *minibatch* do

 if $done_i$ then

$y_i = r_i$

 else

$y_i = r_i + \gamma \max_{a'} \hat{Q}(s'_i, a', \hat{\theta})$

 end

 end

 Calculate the loss $\mathcal{L} = \mathbb{E}_D [y_j - Q(s_j, a_j, \theta)] = 1/N \sum_{i=1}^N (y_i - Q(s_i, a_i, \theta))^2$

 Update Q using the SGD algorithm by minimizing the loss $\mathcal{L}$ (update θ)

 Every C steps, copy weights from Q to $\hat{Q}$ ($\theta \rightarrow \hat{\theta}$)

 end

end

Optimize MSE between Q-network and Q-target



Material

The material illustrated today can be partially found in paragraph 9.8.